



Trabajo con promesas: acceso a ficheros en NodeJS

En este caso práctico vamos a crear una librería para gestionar el almacenamiento de los **gastos** de diferentes **usuarios**.

El objetivo es utilizar una API —en este caso, la API **fs** de NodeJS— que nos permita **practicar con las promesas** en JavaScript. Por tanto, la aplicación no estará optimizada para ofrecer una capa de acceso a datos robusta o profesional.

Los datos se almacenarán en un **fichero** denominado **datos.json**, que tendrá la siguiente estructura:

```
{
  "usuario1": [
    {
      "id": "a91ca2ea-2c3a-442a-8378-059e9f4d9046",
      "descripcion": "Descripción del gasto 1 del usuario 1",
      "valor": 25,
      "fecha": 1655456894000
    }
  ],
  "usuario2": [
    {
      "id": "a51fa3ea-8d39-622b-8368-093a96ef912c",
      "descripcion": "Descripción del gasto 1 del usuario 2",
      "valor": 35.1,
      "fecha": 1655457027000
    },
    {
      "descripcion": "Descripción del gasto 2 del usuario 2",
      "valor": 35.1,
      "fecha": 1655457126000,
      "id": "be488403-b32a-4171-93aa-864651275efa"
    }
  ]
}
```

El fichero almacena un **objeto** con un conjunto de propiedades. Cada **propiedad** del objeto identifica a un **usuario**. El valor de la propiedad es un **array** que almacenará el **listado de gastos**.

La librería deberá crear y exportar **cuatro funciones** que **devuelvan promesas**. A continuación, se indican dichas funciones y las **tareas** que deben realizar:

- **obtenerGastosUsuario(usuario)**. Realizará las siguientes tareas:
 - Leerá el contenido del fichero **datos.json** mediante la función **fs.readFile(NOMBRE_FICHERO)**.
 - Una vez resuelta la promesa que devuelve **fs.readFile**, se convertirán los datos de JSON a un objeto JavaScript.
 - Por último, **devolverá el array** correspondiente a los **gastos del usuario** pasado como parámetro. Si no existe el usuario, se devolverá un **array vacío**.
- **anyadirGastoUsuario(usuario, gasto)**. Realizará las siguientes tareas:
 - Leerá el contenido del fichero **datos.json** mediante la función **fs.readFile(NOMBRE_FICHERO)**.
 - Una vez resuelta la promesa que devuelve **fs.readFile**, se convertirán los datos de JSON a un objeto JavaScript.
 - Se **añadirá el objeto gasto** pasado como parámetro a la lista de gastos del usuario pasado como parámetro. Si el usuario no existe, se creará.



- Por último, escribirán los datos actualizados al fichero mediante la función `fs.writeFile(NOMBRE_FICHERO, DATOS_EN_FORMATO_TEXTO)` y devolverá la promesa generada por `fs.writeFile` [ten en cuenta que esa función escribe únicamente caracteres, no admite objetos JavaScript, por lo que tendrás que hacer la conversión].
- **actualizarGastoUsuario(usuario, gastoId, nuevosDatos)**. Realizará las siguientes tareas:
 - Leerá el contenido del fichero `datos.json` mediante la función `fs.readFile(NOMBRE_FICHERO)`.
 - Una vez resuelta la promesa que devuelve `fs.readFile`, se convertirán los datos de JSON a un objeto JavaScript.
 - Se actualizará el gasto cuyo id sea el `gastoId` pasado como parámetro a la lista de gastos del usuario pasado también como parámetro. Los nuevos datos estarán disponibles en el parámetro `nuevosDatos`.
 - En el caso de que el usuario o el gasto indicados no existan, se generará un error.
 - Por último, escribirá los datos actualizados al fichero mediante la función `fs.writeFile(NOMBRE_FICHERO, DATOS_EN_FORMATO_TEXTO)` y devolverá la promesa generada por `fs.writeFile`.
- **borrarGastoUsuario(usuario, gastoId)**. Realizará las siguientes tareas:
 - Leerá el contenido del fichero `datos.json` mediante la función `fs.readFile(NOMBRE_FICHERO)`.
 - Una vez resuelta la promesa que devuelve `fs.readFile`, se convertirán los datos de JSON a un objeto JavaScript.
 - Se eliminará el gasto cuyo id sea el `gastoId` pasado como parámetro de la lista de gastos del usuario pasado también como parámetro.
 - En el caso de que el usuario indicado no exista, se generará un error.
 - Por último, escribirá los datos actualizados al fichero mediante la función `fs.writeFile(NOMBRE_FICHERO, DATOS_EN_FORMATO_TEXTO)` y devolverá la promesa generada por `fs.writeFile`.



EJEMPLO

En la documentación asociada al Estudio del caso, dispones del archivo `DWEC_U04_A02_CPE_E_archivos.zip`, con el código de partida de la aplicación.

El código está convenientemente comentado. Mediante `TODO`, se indican los sitios donde debes añadir código.

Se proporciona un **conjunto de test** para comprobar la correcta realización de la aplicación. Para ejecutarlos, debes instalar en primer lugar las librerías mediante `npm install`. A partir de ese momento, podrás lanzar los test las veces que sea necesario mediante `npm run test`.

Claves de resolución

- Inspecciona el código proporcionado, incluido el fichero de test.
- Observa que el archivo `package.json` tiene la configuración necesaria para trabajar con módulos ES6.
- Recuerda que las funciones `readFile` y `writeFile` utilizadas devuelven promesas.
- Tienes indicados en los ficheros, mediante la palabra `TODO`, los sitios donde debes añadir código.
- Todas las funciones que realices deberán devolver una promesa.
- Recuerda que el método `then` devuelve a su vez una promesa. Debes utilizarlo para encadenar tareas asíncronas que se ejecuten una a continuación de otra.